

```

import numpy as np
class Adaline:

    def __init__(self, learning_rate=0.01, epochs=50):
        """
        learning_rate : Controls the step size in gradient descent
        epochs         : Number of times the training data is iterated
        """

        self.learning_rate = learning_rate
        self.epochs = epochs

    def fit(self, X, y):
        """
        X : Input feature matrix (n_samples x n_features)
        y : Target output vector
        """

        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0
        self.losses = []

        for epoch in range(self.epochs):

            linear_output = np.dot(X, self.weights) + self.bias
            y_predicted = linear_output

            errors = y - y_predicted

            self.weights += self.learning_rate * np.dot(X.T, errors) / n_samples
            self.bias += self.learning_rate * np.sum(errors) / n_samples
            loss = (errors ** 2).mean() / 2
            self.losses.append(loss)

            print(f"Epoch {epoch + 1}/{self.epochs}, Loss: {loss:.4f}")

    def predict(self, X):
        """
        Predict class labels for input data
        """

        linear_output = np.dot(X, self.weights) + self.bias

        return np.where(linear_output >= 0, 1, -1)

X = np.array([
    [ 1,  1],
    [ 1, -1],
    [-1,  1],
    [-1, -1]
])

y = np.array([1, -1, -1, -1])

model = Adaline()
model.fit(X, y)

predictions = model.predict(X)

print("\nPredicted Labels:", predictions)
print("Actual Labels   :", y)

print("\nFinal Weights:", model.weights)
print("Final Bias   :", model.bias)

```

```

Epoch 1/50, Loss: 0.5000
Epoch 2/50, Loss: 0.4925
Epoch 3/50, Loss: 0.4852
Epoch 4/50, Loss: 0.4781
Epoch 5/50, Loss: 0.4710
Epoch 6/50, Loss: 0.4641
Epoch 7/50, Loss: 0.4574
Epoch 8/50, Loss: 0.4508
Epoch 9/50, Loss: 0.4443
Epoch 10/50, Loss: 0.4379
Epoch 11/50, Loss: 0.4317
Epoch 12/50, Loss: 0.4256

```

```
Epoch 13/50, Loss: 0.4196
Epoch 14/50, Loss: 0.4138
Epoch 15/50, Loss: 0.4080
Epoch 16/50, Loss: 0.4024
Epoch 17/50, Loss: 0.3969
Epoch 18/50, Loss: 0.3915
Epoch 19/50, Loss: 0.3862
Epoch 20/50, Loss: 0.3810
Epoch 21/50, Loss: 0.3759
Epoch 22/50, Loss: 0.3709
Epoch 23/50, Loss: 0.3660
Epoch 24/50, Loss: 0.3612
Epoch 25/50, Loss: 0.3565
Epoch 26/50, Loss: 0.3519
Epoch 27/50, Loss: 0.3474
Epoch 28/50, Loss: 0.3429
Epoch 29/50, Loss: 0.3386
Epoch 30/50, Loss: 0.3343
Epoch 31/50, Loss: 0.3302
Epoch 32/50, Loss: 0.3261
Epoch 33/50, Loss: 0.3221
Epoch 34/50, Loss: 0.3182
Epoch 35/50, Loss: 0.3143
Epoch 36/50, Loss: 0.3106
Epoch 37/50, Loss: 0.3069
Epoch 38/50, Loss: 0.3033
Epoch 39/50, Loss: 0.2997
Epoch 40/50, Loss: 0.2962
Epoch 41/50, Loss: 0.2928
Epoch 42/50, Loss: 0.2895
Epoch 43/50, Loss: 0.2862
Epoch 44/50, Loss: 0.2830
Epoch 45/50, Loss: 0.2799
Epoch 46/50, Loss: 0.2768
Epoch 47/50, Loss: 0.2738
Epoch 48/50, Loss: 0.2708
Epoch 49/50, Loss: 0.2679
Epoch 50/50, Loss: 0.2650

Predicted Labels: [ 1 -1 -1 -1]
Actual Labels    : [ 1 -1 -1 -1]

Final Weights: [0.19749697 0.19749697]
Final Bias    : -0.19749696643123169
```